Week 5 Studio: Events Handling & Managing Routing

Overview: Content

Week 5 Tutorial: Mastering User Interactions in Vue.js

Welcome to Week 5 of our Vue.js journey - so far we have been looking at quite basic concepts, from this week we will build on those to introduce more advanced concepts which are still quite fundamental to web development!

This week, we're diving deep into the heart of interactive web applications: handling events and managing routing / navigation.

First we will recap some of the concepts we have already covered around events and events handling, before we look at some other concepts.

What You'll Learn

By the end of this tutorial, you'll be able to understand more about some concepts we have covered quickly in previous weeks:

Capture User Actions with event handling

Sync Data Effortlessly using two-way data binding

Key Concepts We'll Cover

1. Event Handling

- Implement `v-on` directives to respond to user actions
- Create custom event handlers in Vue components
- Explore event modifiers for fine-tuned control

> Some of this may be familiar from Week 3 - we will be taking your learning in those early weeks and doing some more advanced events-related work in this week's tutorial!

2. Two-Way Data Binding

- Harness the power of `v-model` for seamless data flow
- Dive into the Vue DevTools for real-time data inspection
- Master data binding with various input types



Previously you worked with one-way data binding - now we will explore two-way binding.

3. Routing and Secure Navigation

- While quite basic as far as web development is concerned, many of you will find this complex if you have not come from a web-development background - so prepare to do some extra reading!
- We will look at how to implement single-page applications (SPAs) and introduce routing,
- (D/HD task) We will look at introducing secure routing - conditionally showing web pages to users based on their authentication status.



While this is a D/HD task, mastering this will also help you implement a key Business Requirement for your Assignment - so that's another reason to do this advanced task....

Why This Matters

Mastering events and data binding is crucial for building responsive, user-friendly web applications. These skills form the foundation of interactive Vue.js development and are essential for any aspiring front-end developer.

Let's dive in and start coding!



Dont worry if you don't finish everything in this Studio during the 2.5-hour lab - as long as you finish everything by the end of the week (**Friday 11:55pm**), you are doing great!

Overview: Lab tasks for eFolio

SUBMISSION

- **Submit eFolio tasks 5 & 6 together on Moodle.**

EFOLIO TASK 5.1 (PASS AND CREDIT LEVEL)

WHAT TO SUBMIT (INDIVIDUAL):

- **Screenshot 1**: Modified Library form with evidence of password confirmation validation
- **Screenshot 2**: Using VueJS DevTools to test your components

EVALUATION CRITERIA:

- Screenshot clearly shows error message shown to user when passwords do not match.
- Student demonstrates knowledge of VueJS DevTools to carry out debugging/testing.

EFOLIO TASK 5.2 (DISTINCTION AND HD LEVEL)

Student shows advanced thinking in their ability to implement industry practices around Navigation, Security and Routing.

WHAT TO SUBMIT (INDIVIDUAL):

- A single PDF containing the following files (each file in a new page within the same PDF with header stating what file it is)
  - router/index.js
  - src/App.vue
  - src/views/HomeView.vue
  - Screenshot of App in Localhost (browser) open on Home page
  - Screenshot of App in Localhost (browser) open on About page



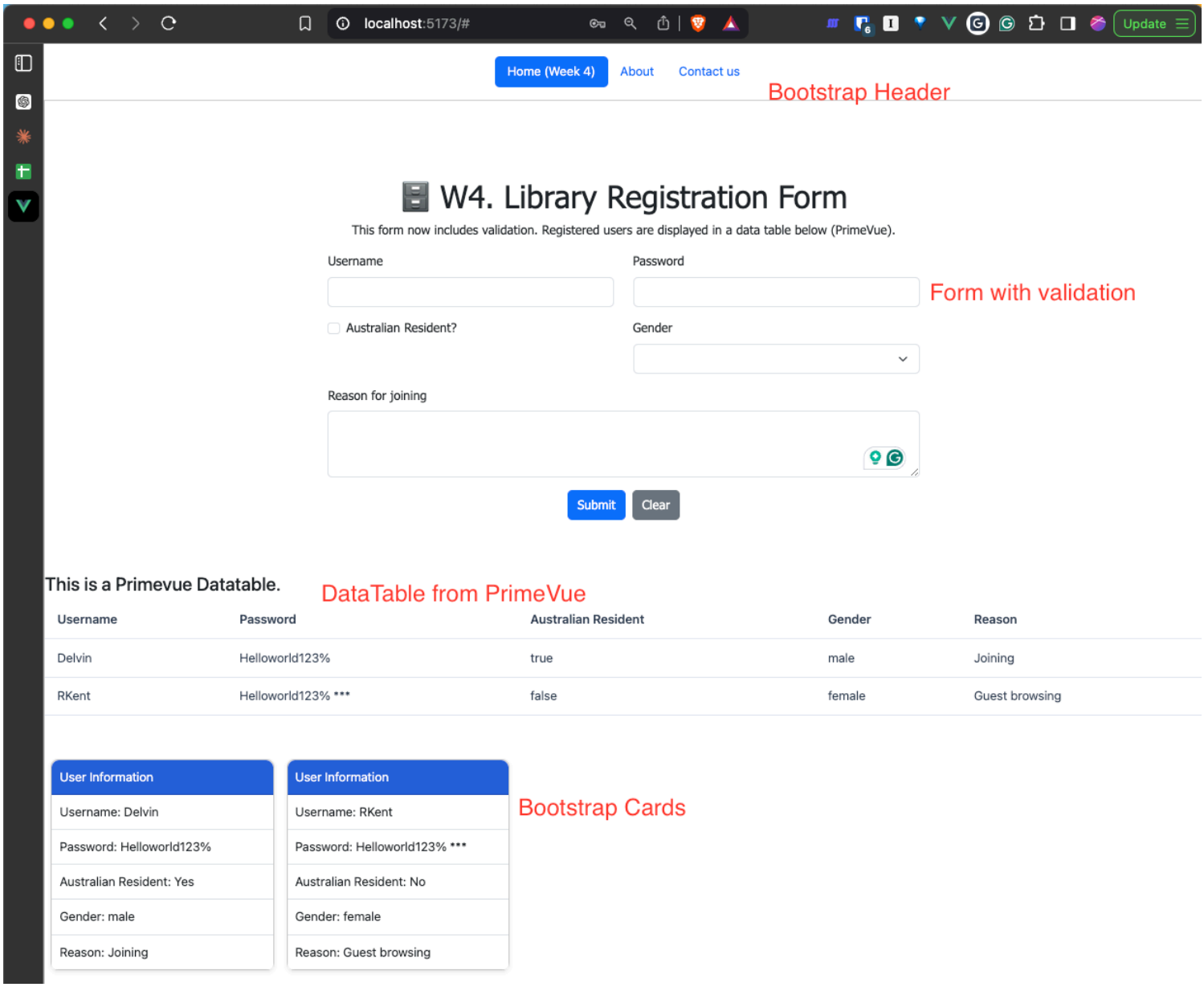
See this article about how to see the commit history of a Github repo: https://docs.github.com/en/pull-requests/committing-changes-to-your-project/viewing-and-comparing-commits/differences-between-commit-views

EVALUATION CRITERIA:

- Screenshot clearly shows working navigation / routes
- Code shows successful learning of key concepts around routing (e.g. conditional routing)

Step 1: Lab Setup (30 mins)

So far you have learnt the skills to build a basic web application like this one here:



Activity 5.1.1 Recreate the starter project

You have two options:

1. Recreate the above website using your existing code (hopefully you have done all the previous weeks content and this should be straightforward for you with a little bit of extra work)
2. **OR** You can download the source code for this from Github here in the **week-4-end** branch.
    1. Clone this repo to your computer
    2. Open it in VS Code
    3. use `git checkout week-4-end` in the VS Code terminal to switch to this branch.

https://github.com/Monash-FIT5032/NoMash-Library

Notice that we have added a BHeader component.

```
<template>
  <!-- Using Bootstrap's Header template (starter code) -->
  <!-- https://getbootstrap.com/docs/5.0/examples/headers/ -->
  <div class="container">
    <header class="d-flex justify-content-center py-3">
      <ul class="nav nav-pills">
        <li class="nav-item">
          <a href="#" class="nav-link active" aria-current="page">Home (Week 5)</a>
        </li>
        <li class="nav-item"><a href="#" class="nav-link">About</a></li>
        <li class="nav-item"><a href="#" class="nav-link">Contact us</a></li>
      </ul>
    </header>
  </div>
</template>
```

If you use starter code then don't forget to update PrimeVue library (check notes under Week 4)!

```
npm install primevue
npm install @primeuix/themes
```

```
import PrimeVue from 'primevue/config';
import Aura from '@primeuix/themes/aura';
```

[Updated by Russel, August 2026]

If you have never worked with Github branches before - this is your chance to learn. You will absolutely struggle in industry if you don't know how to use Github branches!!

Prefer to learn from a video?

▶ 扩大

Instructions on using Github branches below:

Step 2: Events Recap (10 mins)

Event 1: @submit.prevent

Line 85 of LibraryRegistrationForm.vue:

```
<form @submit.prevent="submitForm">
```

In Vue.js, we use the `@submit.prevent` event to handle form submission events. The `@submit` directive allows us to listen for the form's submit event, and the `.prevent` modifier ensures that `event.preventDefault()` is called, preventing the default action of submitting the form.

This is particularly useful when we want to manage form submissions with JavaScript, such as validating the form data, without causing the page to reload. By using `@submit.prevent="submitForm"`, we can trigger the `submitForm` method defined in our Vue component, allowing us to implement custom form handling logic while preventing the default form submission behaviour.

Event 2: @blur

Line 93 of LibraryRegistrationForm.vue:

```
@blur="() => validateName(true)"
```

In Vue.js, the `@blur="() => validateName(true)"` syntax binds a blur event handler to an input element, which triggers when the element loses focus. The handler is an arrow function that calls the `validateName` function with `true` as an argument. This setup is typically used for input validation; **when the user moves away from the input field, the blur event fires**, invoking `validateName(true)` to validate the input.

Event 3: @input

Line 94 of LibraryRegistrationForm.vue:

```
@input="() => validateName(false)"
```

This syntax binds an input event handler to an input element, which triggers whenever the input's value changes. The handler is an arrow function that calls the `validateName` function with `false` as an argument. This setup is typically used for real-time input validation; as the user types, the input

event fires, invoking `validateName(false)` to validate the input continuously.

Event 4: @click

Line 145 of LibraryRegistrationForm.vue:

```
<button type="button" class="btn btn-secondary" @click="clearForm">Clear</button>
```

Q: What is the difference between `v-on:click` and `@click`?

▶ 扩大

In Vue.js, we use the `@click` event to handle click events on elements such as buttons. In this example, `<button type="button" class="btn btn-secondary" @click="clearForm">Clear</button>`, the `@click` directive listens for a click event on the button. When we click the button, the `clearForm` method is called, which is defined in our Vue component. This method typically contains the logic to clear the form fields, resetting them to their default values. By using the `@click` event handler, we can easily attach custom functionality to the button click, making our form interactive and user-friendly.

Let's take a look inside clearForm,

```
const clearForm = () => {
  formData.value = {
    username: '',
    password: '',
    isAustralian: false,
    reason: '',
    gender: ''
  }
}
```

What's going on here when the user clicks the "Clear" button?

▶ 扩大

🗄️ W4. Library Registration Form

This form now includes validation. Registered users are displayed in a data table below (PrimeVue).

Username

helloworld

Password

••••••••••••••••••••

Australian Resident?

Gender

Female

Reason for joining

blahblah blah

Submit Clear

Step 3: Adding more events (30 mins)

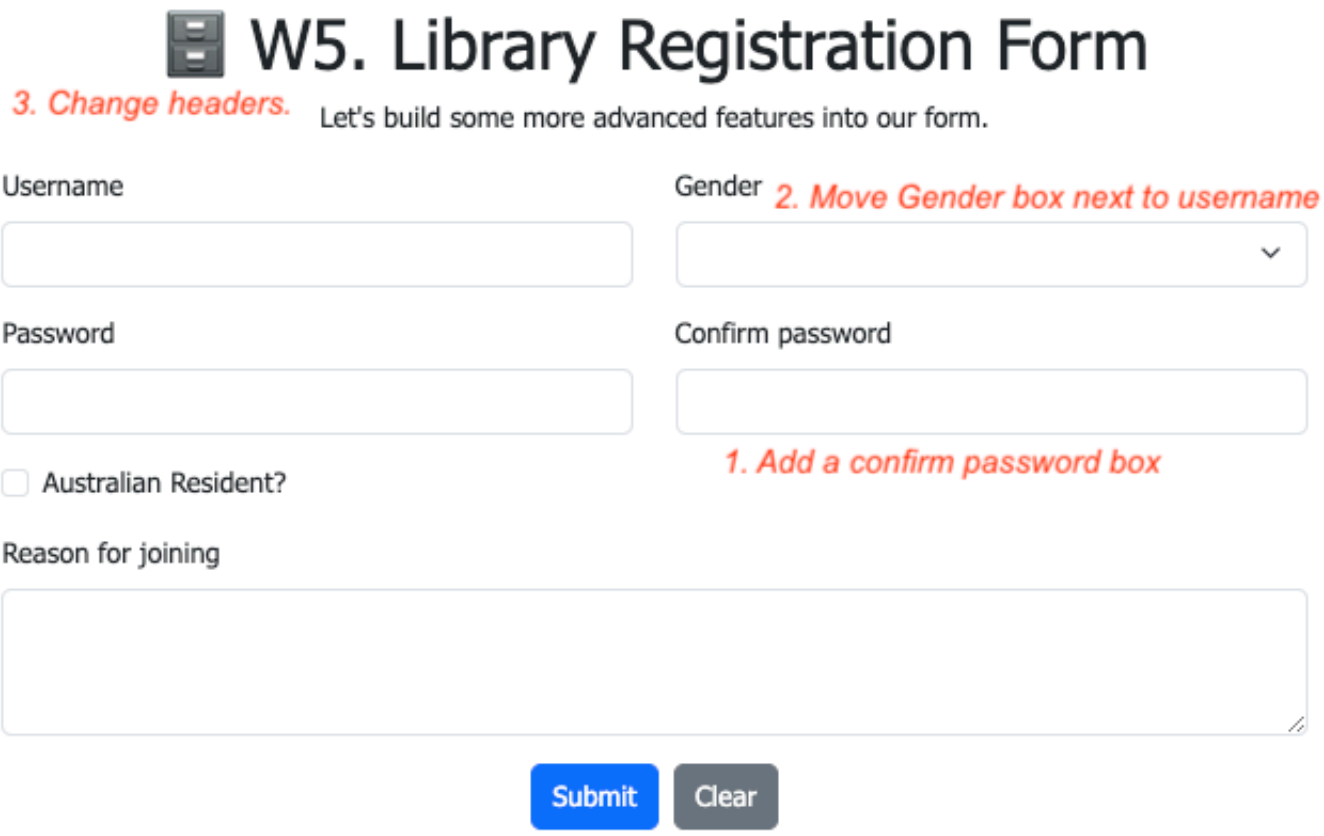
We want to modify the Library sign-up page to include a password confirmation form.

When building the signup page for our library web application, including a password confirmation box is more than just a common practice—it's a crucial element for both security and user experience. **But why?**

▶ 扩大

Activity 5.3.1: Setup the Form

Using your knowledge of Bootstrap and the screenshot below, change your form to make it look like this, adding a confirm password input field.



The only change I've made for the confirm-password is change the v-model and changed the name, but apart from that the code remains unchanged.

```
<div class="col-md-6 col-sm-6">
  <label for="confirm-password" class="form-label">Confirm password</label>
  <input
    type="password"
    class="form-control"
    id="confirm-password"
    v-model="formData.confirmPassword"
  />
  <div v-if="errors.password" class="text-danger">{{ errors.password }}</div>
</div>
```

Let's go to the formData and add a variable for confirmPassword.

```
const formData = ref({
  username: '',
  password: '',
  confirmPassword: '',
  isAustralian: false,
  reason: '',
  gender: ''
})
```

Also modify the errors constant to support confirmPassword:

```
const errors = ref({
  username: null,
  password: null,
  confirmPassword: null,
  resident: null,
  gender: null,
  reason: null
})
```

And finally, lets modify the error div for confirmPassword. Previously, it looked like this:

```
<div v-if="errors.password" class="text-danger">{{ errors.password }}</div>
```

Now, we can change it to:

```
<div v-if="errors.confirmPassword" class="text-danger">
                {{ errors.confirmPassword }}
              </div>
```

Remember how we used @blur and @input for the previous elements for validation, we need to use

a similar approach here for confirmPassword.

Reminder about key differences:

- **Timing**: `@blur` fires when the element loses focus, while `@input` fires immediately as the input value changes.
- **Use Case**: Use `@blur` for actions that should occur after the user finishes interacting with the input, such as final validation or formatting. Use `@input` for actions that should occur in real-time as the user types, such as live search suggestions, instant feedback, or updating data models.



It might be annoying for the user if we are constantly reminding them that the passwords do not match before they have finished typing, so we are only interested in using @blur instead of @input.

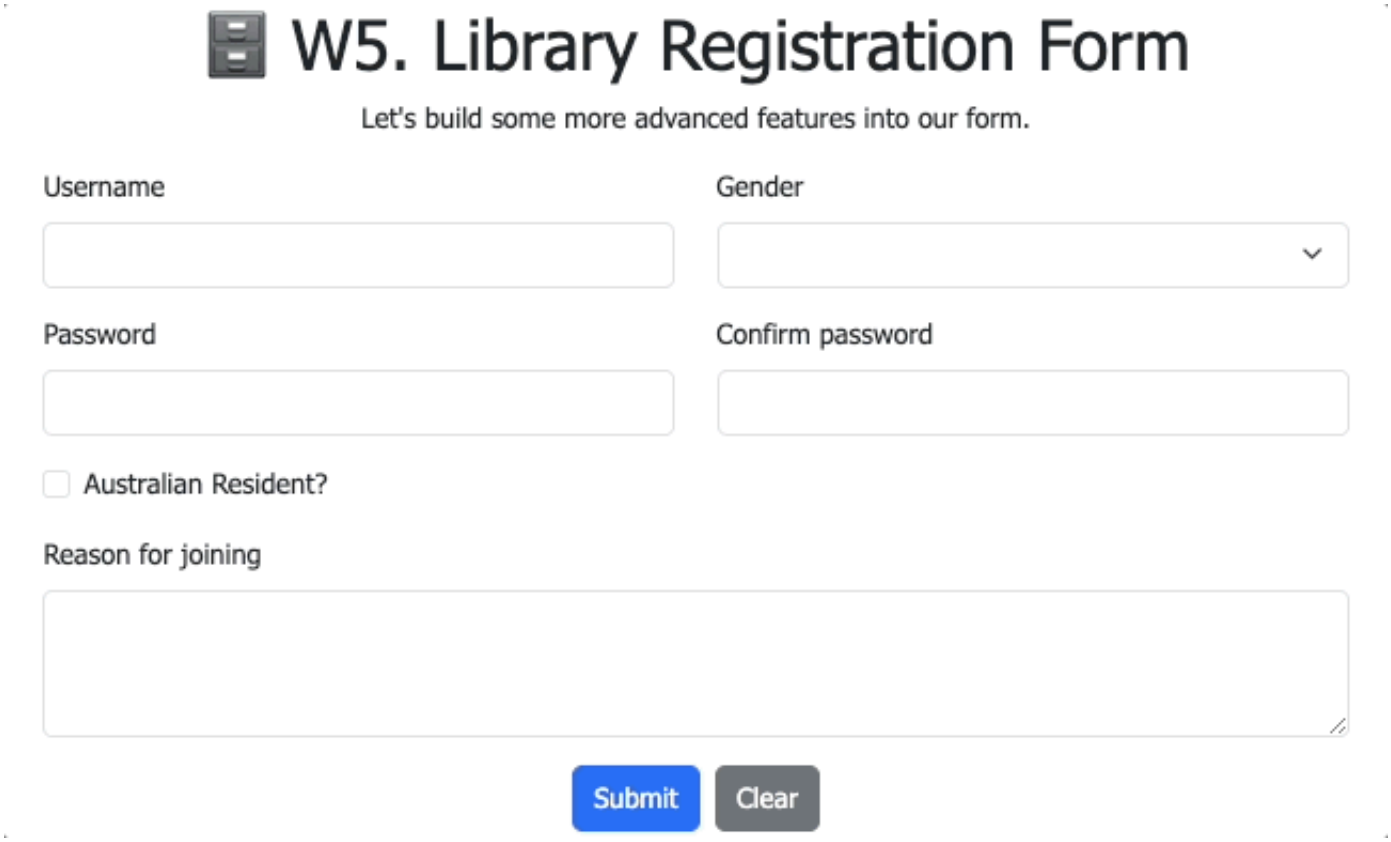
We are ready now to write a function that does error validation:

```
/**
 * Confirm password validation function that checks if the password and confirm passwo
 * @param blur: boolean - If true, the function will display an error message if the p
 */
const validateConfirmPassword = (blur) => {
  if (formData.value.password !== formData.value.confirmPassword) {
    if (blur) errors.value.confirmPassword = 'Passwords do not match.'
  } else {
    errors.value.confirmPassword = null
  }
}
```

And let's call this function in our confirmPassword input:

```
<div class="col-md-6 col-sm-6">
    <label for="confirm-password" class="form-label">Confirm password</label>
    <input
        type="password"
        class="form-control"
        id="confirm-password"
        v-model="formData.confirmPassword"
        @blur="() => validateConfirmPassword(true)"
    />
    <div v-if="errors.confirmPassword" class="text-danger">
        {{ errors.confirmPassword }}
    </div>
</div>
```

Success, lets see how confirm password feature in action.



Notice how the validation only happens when the confirm password field loses focus?

Activity 5.3.2 Improve validation for "Reason for joining"

Modify the Reason for joining input to have some input text validation. If the text contains the word 'friend', then show a green message underneath telling the user, "Great to have a friend".

🗄️ W5. Library Registration Form

Let's build some more advanced features into our form.

Username

Gender

Password

Confirm password

••••

Passwords do not match.

Australian Resident?

Reason for joining

Reason must be at least 10 characters

Submit Clear

Screenshot:

Reason for joining

my name is Mary and i heard about this from my friend

Great to have a friend

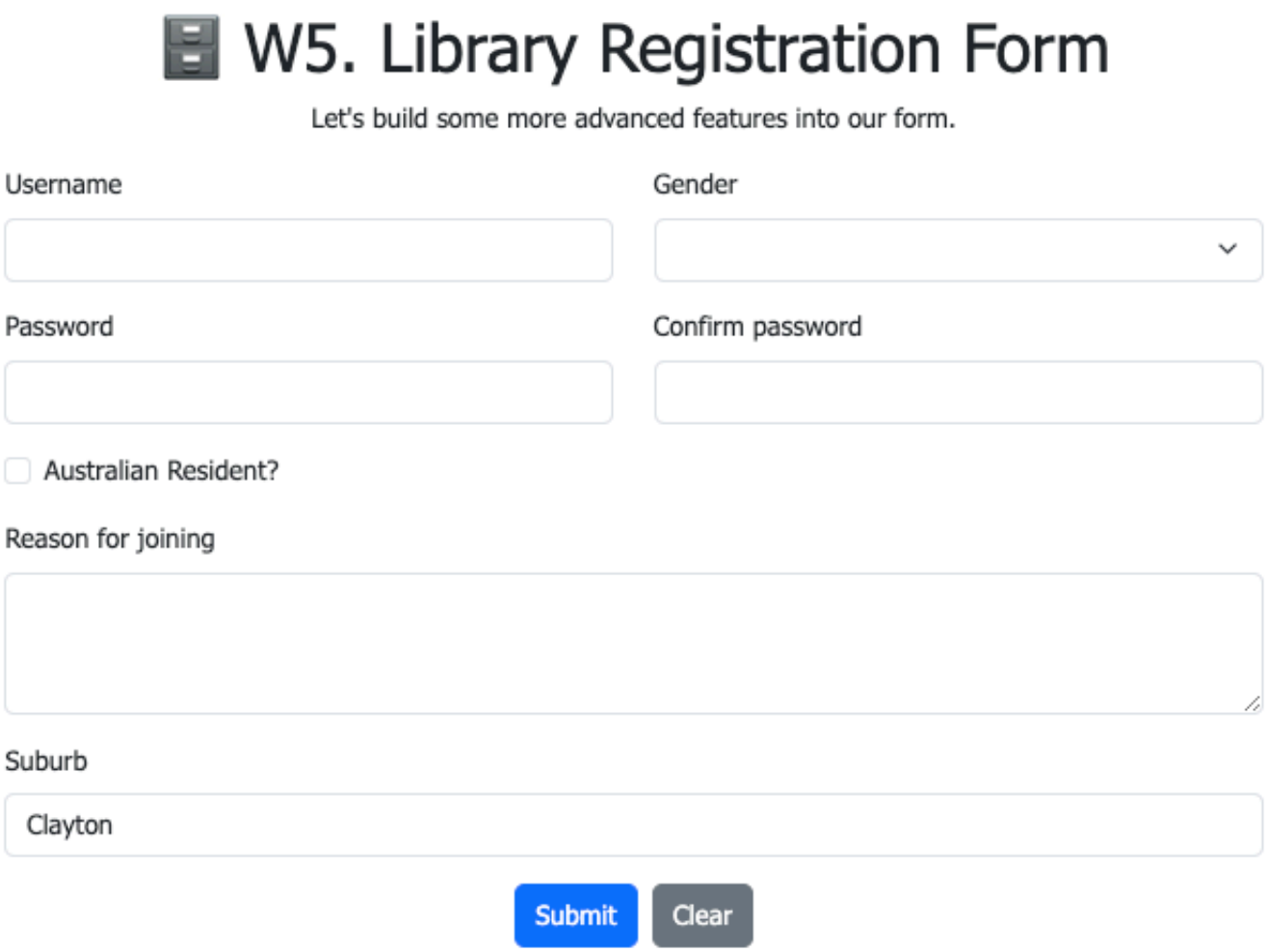
Step 4: Benefit of using v-model (20 mins)

You might have noticed how we used v-model quite generously in this library application.

Vue.js's `v-model` directive simplifies form input handling by providing two-way data binding between form elements and the component's data.

While v-bind allows one-way data binding, v-model allows 2-way binding. Let's see the difference in action.

I've modified my form to include a Suburb field:



The input I've added to my form is as follows:

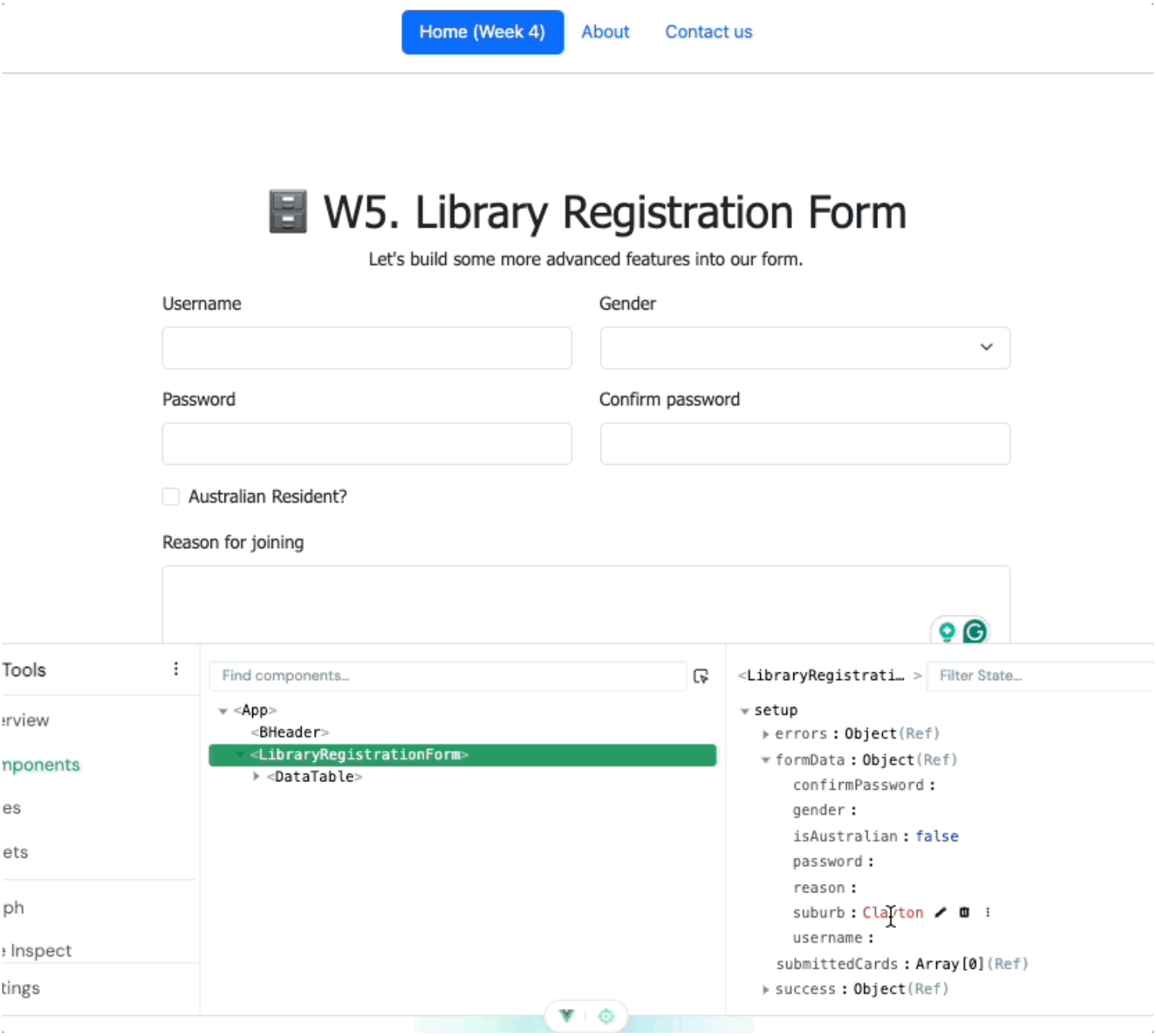
```
<div class="mb-3">
            <label for="reason" class="form-label">Suburb</label>
            <input type="text" class="form-control" id="suburb" v-bind:value="formData
          </div>
```

And you can see that the v-bind is used to bind the value of this input to a constant called suburb

inside my script's formData:

```
const formData = ref({
  username: '',
  password: '',
  confirmPassword: '',
  isAustralian: false,
  reason: '',
  gender: '',
  suburb: 'Clayton'
})
```

If you open your Vue DevTools and play around with changing the form values you will notice something curious:



Did you notice what's going on?

Because we used one-way data-binding on the suburb, no matter what changes you make on the front-end the changes are not filtered back to the data source (formData).

Step 5: Adding a Router (30 mins)

So far we have had a boring one-page application. Let's change that.

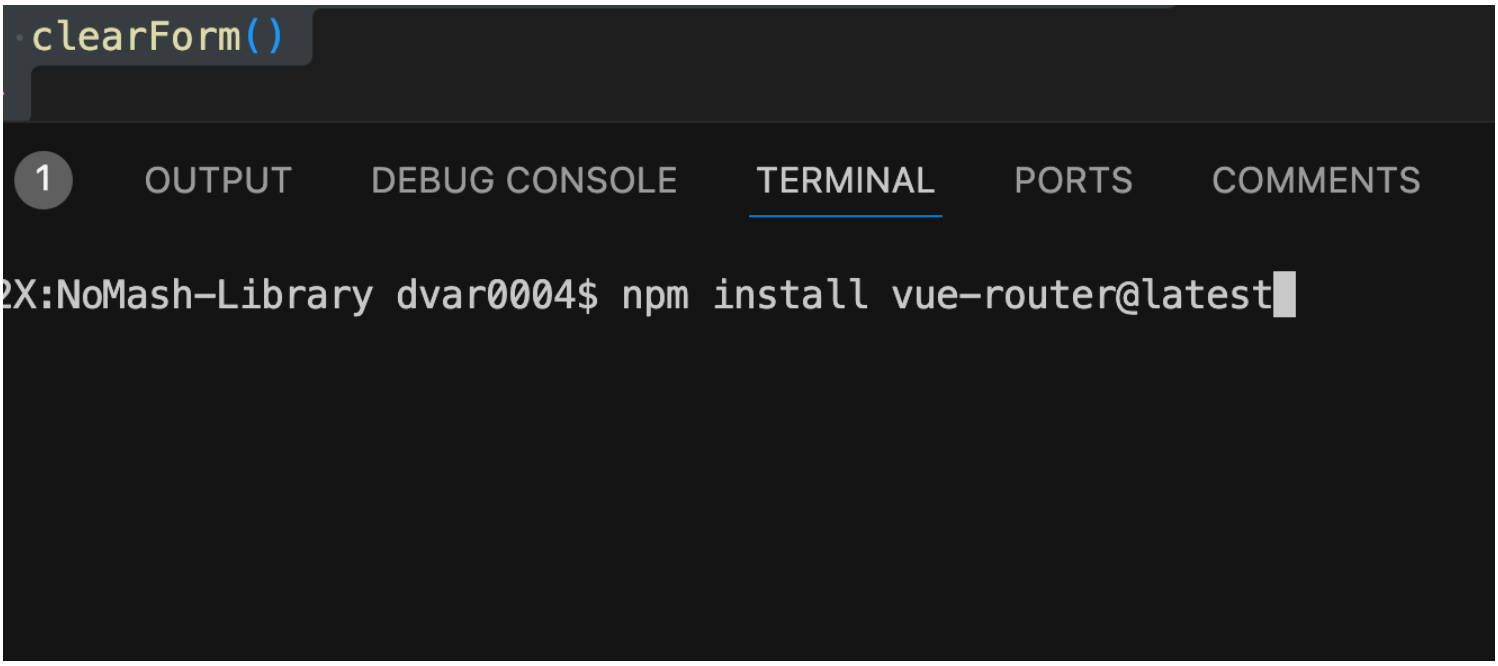
In this activity, we'll implement Vue Router to create a multi-page experience in our single-page application. We'll create two views: the existing registration form and a new "About" page for our library. We will also add the previously created page demonstrating our VueJS knowledge in computed properties as an additional page.

Activity 5 Installing and Setup of Router

5.1: Install Router

First, we need to install Vue Router. In your project directory, run using the Terminal in VS Code:

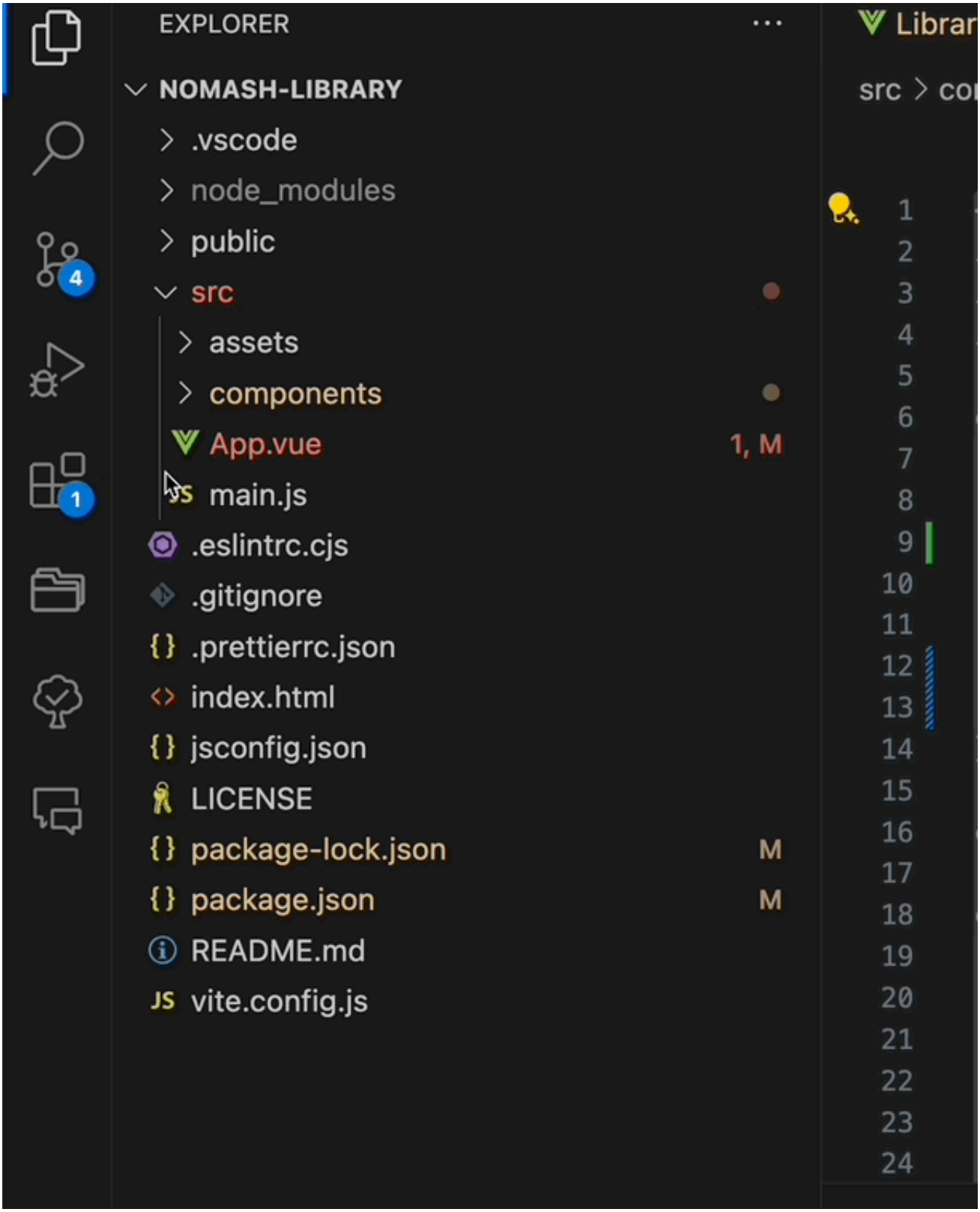
```
npm install vue-router@latest
```



⚠ If any vite issue, use npm install vue-router@4

In VueJS, each page is called a **View**.

Let's create a views folder to store all the different pages.



5.2. Creating Views

Create two new files in your `src/views` directory (create this directory if it doesn't exist):

- `HomeView.vue`: This will contain our existing registration form. Simply rename your Form.vue to HomeView.vue and move this to the src/views directory.
- `AboutView.vue`: This will be a new page with information about the library.

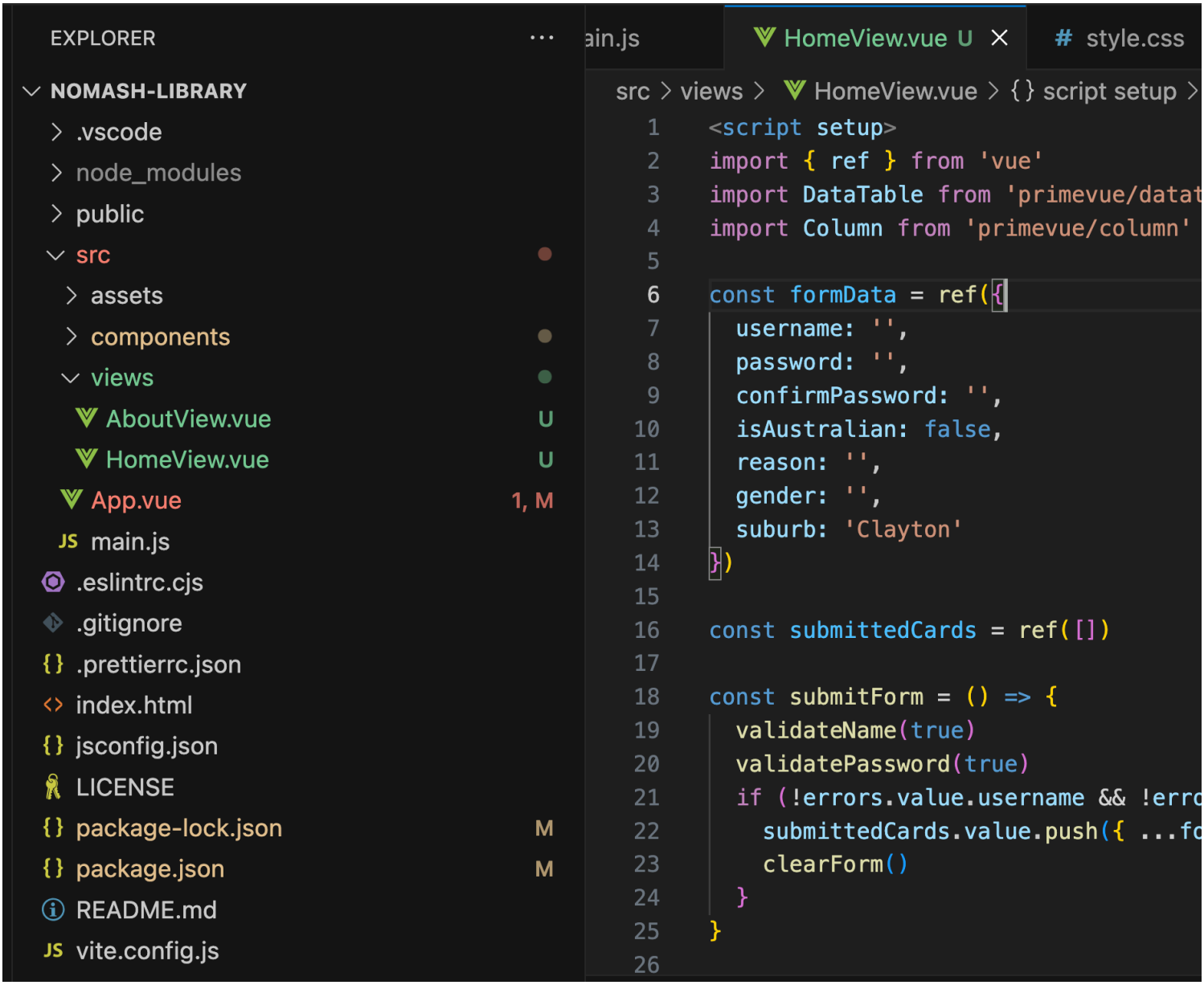
Here are the contents of the AboutView.vue page

```
<template>
  <div class="about">
    <h1>About Our Library</h1>
    <p>Welcome to our digital library! We're dedicated to providing a vast collection
  </div>
</template>

<script setup>
// No script needed for now
</script>

<style>
</style>
```

Hopefully you are all set up like this:



5.3. Now it is time to setup the router.

Create a file called index.js inside src/router/index.js.

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import AboutView from '../views/AboutView.vue'

const routes = [
  {
    path: '/',
    name: 'Home',
    component: HomeView
  },
  {
    path: '/about',
    name: 'About',
    component: AboutView
  }
]

const router = createRouter({
  history: createWebHistory(),
  routes
})

export default router
```

5.4: Modify Main App Component

Update your main app component (likely `App.vue`) to use the router.

For example, before our App.vue template looked like this:



Let's modify this by commenting out the LibraryForm to add the router view instead..

```
<router-view></router-view>
```

```vue
<template>
  <div class="main-container">
    <header>
      <BHeader />
    </header>

    <main class="main-box">
      <!-- <LibraryRegistrationForm /> -->
      <!-- <JSONLab /> -->
      <router-view></router-view>
    </main>
  </div>
</template>
```

5.5. Modify the navigation menu

Open your Header component (BHeader.vue), this is what it looks like currently:

src > components > BHeader.vue > { } template

```vue
<template>
  <!-- Using Bootstrap's Header template (starter code) -->
  <!-- https://getbootstrap.com/docs/5.0/examples/headers/ -->
  <div class="container">
    <header class="d-flex justify-content-center py-3">
      <ul class="nav nav-pills">
        <li class="nav-item">
          <a href="#" class="nav-link active" aria-current="page">Home (Week 5)</a>
        </li>
        <li class="nav-item"><a href="#" class="nav-link">About</a></li>
        <li class="nav-item"><a href="#" class="nav-link">Contact us</a></li>
      </ul>
    </header>
  </div>
</template>
```

We need to change the navigation so that it is done using **router-link**.

```
<template>
  <!-- Using Bootstrap's Header template (starter code) -->
  <!-- https://getbootstrap.com/docs/5.0/examples/headers/ -->
  <div class="container">
    <header class="d-flex justify-content-center py-3">
      <ul class="nav nav-pills">
        <li class="nav-item">
          <router-link to="/" class="nav-link" active-class="active" aria-current="pag
            >Home (Week 5)</router-link
          >
        </li>
        <li class="nav-item">
          <router-link to="/about" class="nav-link" active-class="active">About</route
        </li>
      </ul>
    </header>
  </div>
</template>
```

What's changed?

To modify your Bootstrap-enabled navigation menu to include Vue Router, you need to replace the `<a>` tags with `<router-link>` components provided by Vue Router. The `<router-link>` component allows you to define navigation links that will update the URL and keep the application in sync with the browser history.

5.6. Update Main.js

Modify your `main.js` file to use the router:

```
import 'bootstrap/dist/css/bootstrap.min.css'

import { createApp } from 'vue'
import App from './App.vue'
import router from './router'

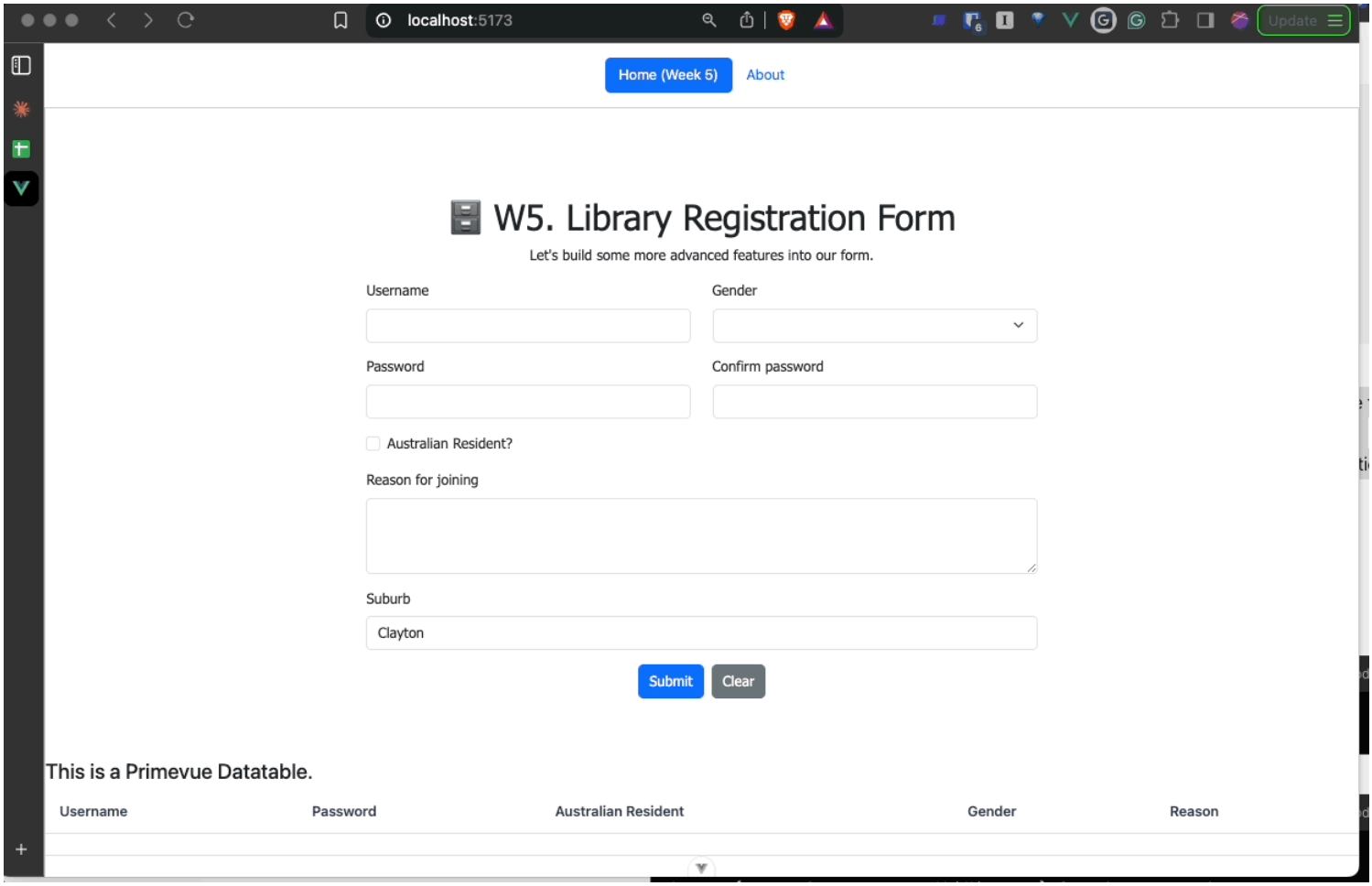
import PrimeVue from 'primevue/config'
import Aura from '@primevue/themes/aura'


const app = createApp(App)
app.use(PrimeVue, { theme: { preset: Aura } })
app.use(router)

app.mount('#app')
```

5.7: Test Your Router

Run your application and navigate between the Home and About pages using the links in the navigation bar.



5.8: Clean-up!

Lets clean up the CSS so that our website has a consistent look and feel.

All we need to do is move the CSS currently used for styling the container from the LibraryForm and moving this to App.vue, so that all pages in our website look and feel the same. And use consistent use of Bootstrap elements to show the header

Try this yourself.

Expected output:

Home (Week 5) About

🗄️ W5. Library Registration Form

Let's build some more advanced features into our form.

Username

Gender

Password

Confirm password

Australian Resident?

Reason for joining

Suburb

Clayton

Submit Clear

This is a Primevue Datatable.

Username	Password	Australian Resident	Gender	Reason

Step 6: Custom Routing (D/HD Task) (30 mins)

Activity 6.1: Creating a Secure Members Area in Your Library App

Learning Objective

1. Implement a login system and secure members area in your library application using Vue Router and conditional rendering.
2. Use **Vue's Router documentation** to successfully implement intermediate-level routing functionality

Background

Not all pages of a website should be publicly accessible. In this activity, you'll create a login page and restrict access to certain parts of your library application to logged-in users only.

Requirements

- Create a login page with username and password fields.
- Create an access-denied page to show unauthenticated users.
- Implement a simple authentication system (use hardcoded credentials for now).
- Restrict access to certain routes (e.g., About page) to authenticated users only.
- Redirect unauthenticated users to the login page when they try to access restricted routes.
- Show a logout option for authenticated users.

Hints and Guidelines

1. Login Page

- Create a new view for the login page called LoginView.vue.
- Use `v-model` for two-way binding on input fields.
- Implement a method to handle form submission.

2. Authentication

- Validate the user's username/password and store the authentication state (e.g., `isAuthenticated`) in your app. Hint: use global `isAuthenticated` state to share across your app or manage global state using Pinia store.

- Consider using Vue's `ref()` for managing this state.
- For this exercise, hardcode a username and password for verification.

3. Authenticated users (Successful login)

- if user enters the right credentials, allow them to see the About page

4. Redirecting Unauthenticated Users

- If user is not logged in (check hardcoded credentials), then do not allow them to open the About page.
- *Hint: check out navigation guards for one way of doing this.*

5. Conditional Rendering

- Use `v-if` and `v-else` directives to show different content based on authentication status.
- Consider creating a navigation component that changes based on whether the user is logged in.

6. Logout Functionality

- Use Vue Router to navigate back to the login page after logout.

Post-Lab Advanced Tasks: Using Navigation Guards in Vue Router (60 mins)

Read the official documentation on VueRouter (Composition API - reviewed by Xuanao Zhao)

https://router.vuejs.org/guide/advanced/composition-api.html#navigation-guards

Navigation guards in Vue Router provide a way to control access to routes based on certain conditions. They are functions that run during the navigation process and can decide whether to allow or prevent navigation. There are several types of navigation guards in Vue Router:

- **Global Guards**: These are guards that apply to all routes.
- **Per-Route Guards**: These are guards defined within route configurations.
- **In-Component Guards**: These are guards defined within the components themselves.

Global Guards

Global guards apply to all route changes. There are three types of global guards:

- `beforeEach`: Called before every navigation.
- `beforeResolve`: Called before navigation is confirmed.
- `afterEach`: Called after every navigation is confirmed.

Example of global `beforeEach` guard:

```
import { createRouter, createWebHistory } from 'vue-router';
import Home from '../views/Home.vue';
import About from '../views/About.vue';

const routes = [
  { path: '/', name: 'Home', component: Home },
  { path: '/about', name: 'About', component: About }
];

const router = createRouter({
  history: createWebHistory(process.env.BASE_URL),
  routes
});

router.beforeEach((to, from, next) => {
  // Perform logic before every route change
  if (to.name !== 'Home') {
    // For example, redirect to home if trying to access any other route
    next({ name: 'Home' });
  } else {
    next();
  }
});

export default router;
```

Per-Route Guards

Per-route guards are defined directly in the route configuration object.

Example of per-route `beforeEnter` guard:

```
const routes = [
  {
    path: '/',
    name: 'Home',
    component: Home
  },
  {
    path: '/about',
    name: 'About',
    component: About,
    beforeEnter: (to, from, next) => {
      // Perform logic before entering the About route
      if (someCondition) {
        next();
      } else {
        next(false); // Cancel the navigation
      }
    }
  }
];
```

In-Component Guards

In-component guards are defined as component options within the components themselves. There are three types of in-component guards:

- **`beforeRouteEnter`**: Called before the route is confirmed, but the component instance is not yet available.
- **`beforeRouteUpdate`**: Called when the route changes, but the same component instance is being reused.
- **`beforeRouteLeave`**: Called when navigating away from the route.

Example of in-component guards:

```
<template>
    <div> About Page </div>
</template>

<script setup>
// note the "setup" attribute in <script setup> tag above this line.
// adding a "setup" text switches to CompositionAPI

import { onBeforeRouteLeave, onBeforeRouteUpdate } from 'vue-router';

onBeforeRouteLeave((to, from, next) => { /* ... */ });
onBeforeRouteUpdate((to, from, next) => { /* ... */ });

function someMethods() { /*...*/ }
</script>
```

Summary

Navigation guards in Vue Router allow you to control access and behavior during navigation. They provide hooks for:

- Performing actions before and after navigation (global guards).
- Controlling access to specific routes (per-route guards).
- Handling navigation events within components (in-component guards).

By using these guards, you can implement authentication checks, data fetching, permission handling, and more complex navigation logic in your Vue.js applications.